Case Study 1: Progressive Water Tariff Analysis

1. Problem statement

Develop a Python program to calculate a water bill according to the quantity of water consumed. The program must apply progressive tariff rates to different consumption slabs, calculate the total bill, and display the applicable charges.

2. Objectives

- **Calculate the water bill based on water consumption.**
- **Understand conditional statements and arithmetic operations.**
- **Apply different rates to different consumption slabs.**
- **Validate input and display a clear billing summary.**

3. Assumptions

For this assignment, use the following illustrative tariff:

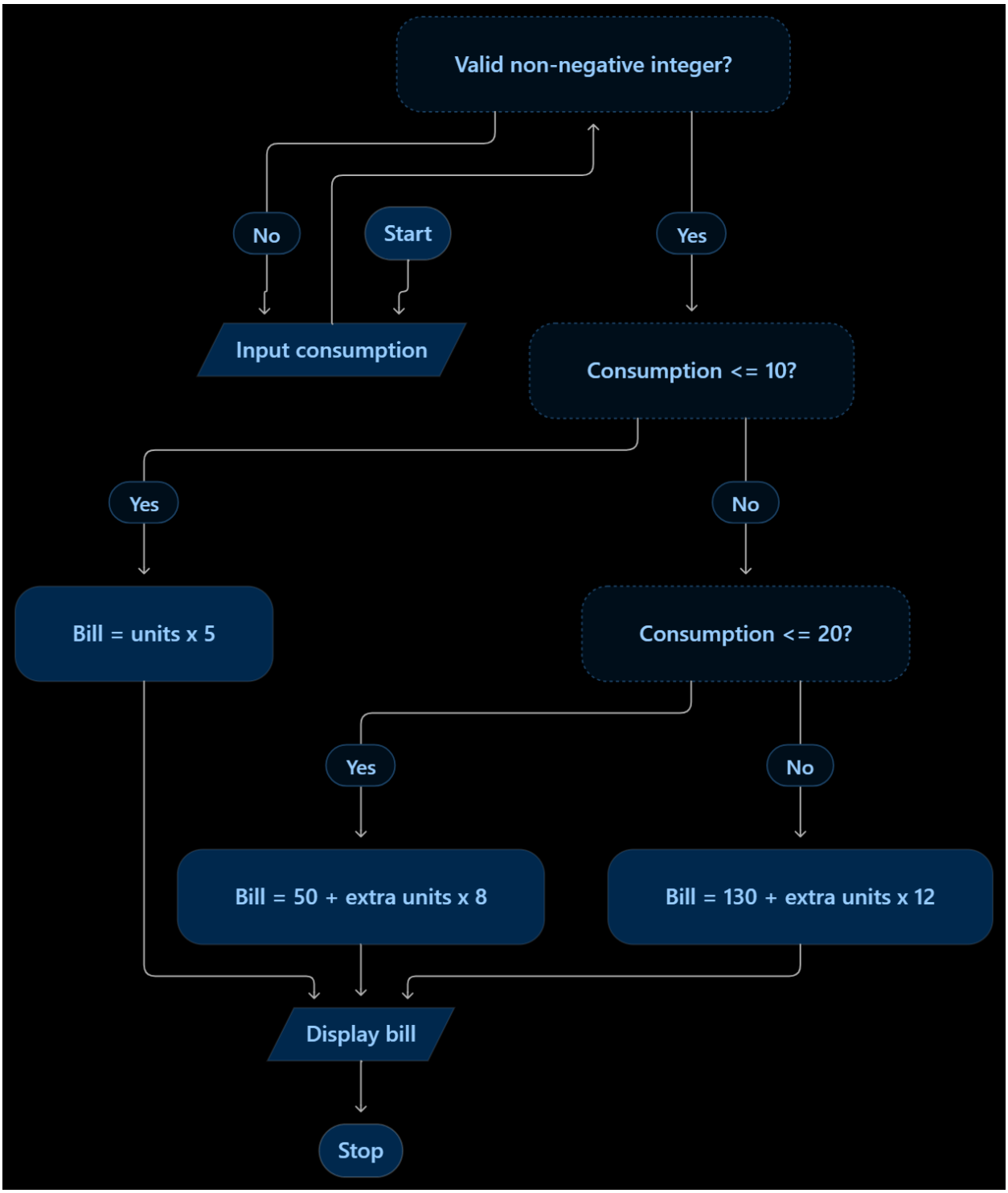
Consumption slab	Rate
First 10 units	₹5 per unit
Next 10 units	₹8 per unit
Above 20 units	₹12 per unit

- **Consumption is entered as a non-negative whole number.**
- **Charges are progressive: each rate applies only to the units within its slab.**
- **No fixed charges, taxes, or penalties are included.**

4. Algorithm

1. **Start.**
2. **Input water consumption.**
3. **Validate that consumption is a non-negative whole number.**
4. **If consumption is at most 10, charge all units at ₹5.**
5. **If consumption is between 11 and 20, charge the first 10 units at ₹5 and the remaining units at ₹8.**
6. **If consumption exceeds 20, charge the first 10 units at ₹5, the next 10 at ₹8, and the remaining units at ₹12.**
7. **Display consumption and total bill.**
8. **Stop.**

5. Flowchart:


Valid non-negative integer?
No
Start
Yes
Input consumption
Consumption <= 10?
Yes
No
Bill = units x 5
Consumption <= 20?
Yes
No
Bill = 50 + extra units x 8
Bill = 130 + extra units x 12
Display bill
Stop

6. Pseudocode

```
BEGIN
  INPUT units

  IF units <= 10
    bill = units * 5
  ELSE IF units <= 20
    bill = 10 * 5 + (units - 10) * 8
  ELSE
    bill = 10 * 5 + 10 * 8 + (units - 20) * 12
  END IF

  DISPLAY units, bill
END
```

7. Python source code and output

```python
while True:
    try:
        units = int(input("Enter water consumption in units: "))
        if units >= 0:
            break
        print("Consumption cannot be negative.")
    except ValueError:
        print("Enter a valid whole number.")

if units <= 10:
    bill = units * 5
elif units <= 20:
    bill = 10 * 5 + (units - 10) * 8
else:
    bill = 10 * 5 + 10 * 8 + (units - 20) * 12

print("\n--- WATER BILL ---")
print("Water consumption:", units, "units")
print("Total bill: Rs.", format(bill, ".2f"))
```

```
'c:\Users\Shaik Sania\OneDrive\Desktop\python\activity1.py'
Enter water consumption in units: 10

--- WATER BILL ---
Water consumption: 10 units
Total bill: Rs. 50.00
PS C:\Users\Shaik Sania\OneDrive\Desktop\python>
```

8. Test cases

Test	Input	Expected bill
1	0 units	₹0
2	10 units	₹50

3	**15 units ₹90**
4	**25 units ₹190**
5	**-5 units Rejected**

9. Result and conclusion

The program calculates water bills correctly using progressive tariff slabs. It demonstrates how conditional statements and arithmetic operations can solve a practical billing problem.

Case Study 2: Password Policy Validation Engine

1. Problem statement

- **Develop a Python program that validates a password according to predefined security rules. The program must check its length and determine whether it contains uppercase letters, lowercase letters, digits, and special characters. It must display which requirements are satisfied and whether the password is valid.**

2. Objectives

- **Understand string handling and character checking.**
- **Use loops to examine individual characters.**
- **Apply conditional statements to enforce password rules.**
- **Validate input and display meaningful results.**

3. Assumptions

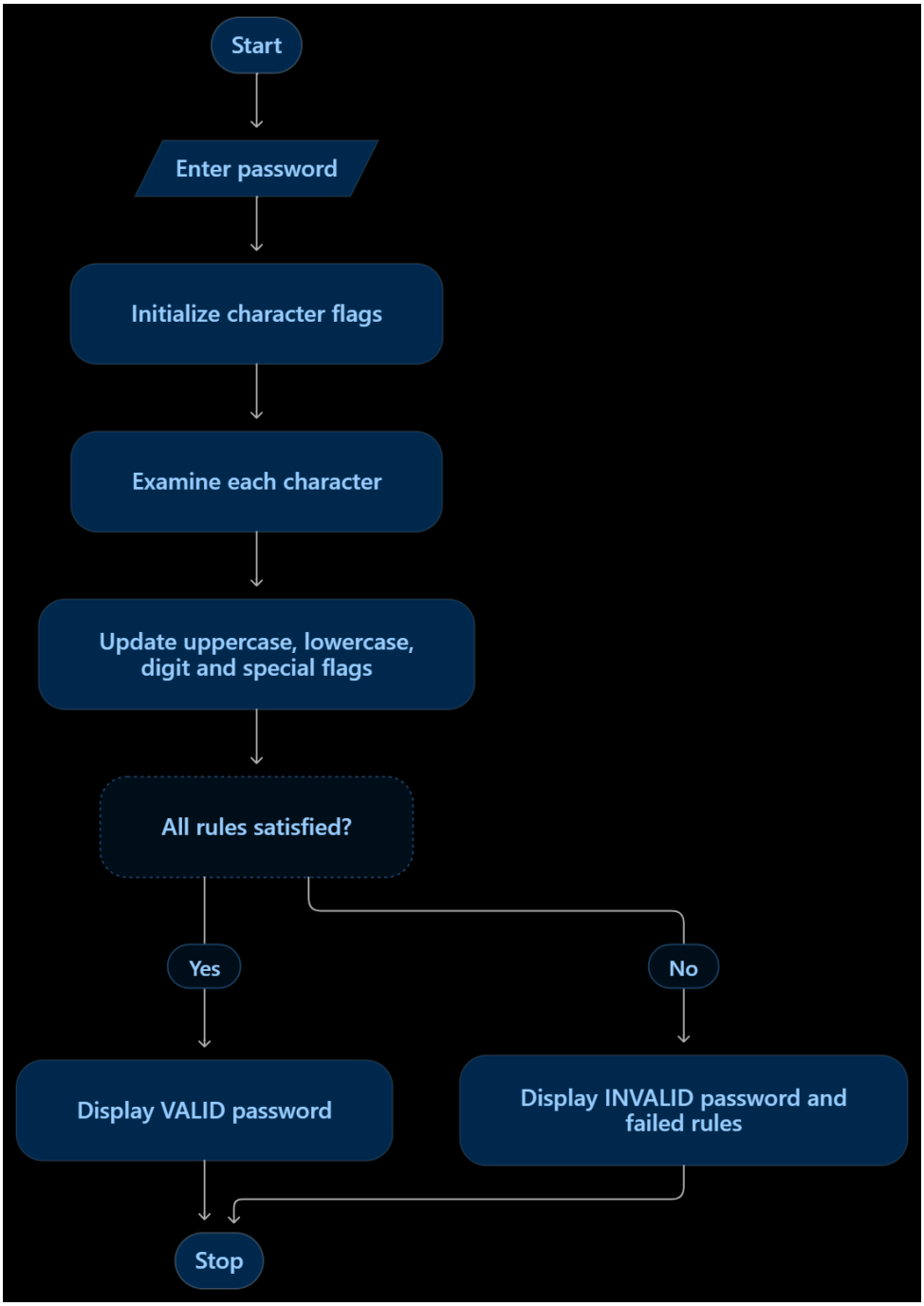
- **A valid password must:**
- **Contain at least 8 characters.**
- **Contain at least one uppercase letter.**
- **Contain at least one lowercase letter.**
- **Contain at least one digit.**
- **Contain at least one special character from !@#$%^&*.**
- **Spaces are not accepted. This is an educational validator, not a complete production security system.**

4. Algorithm

- **Start.**
- **Read the password.**
- **Initialize four Boolean flags to False.**
- **Examine each character using a loop.**

- **Update the relevant flag for uppercase, lowercase, digit, or special character.**
- **Check the minimum length and space restriction.**
- **Display each rule's status.**
- **Display whether the password is valid.**
- **Stop.**

5. Flowchart



6. Pseudocode

- **BEGIN**
- **INPUT password**
-

- **SET upper, lower, digit, special = FALSE**
-
- **FOR each character IN password**
- **IF uppercase THEN upper = TRUE**
- **IF lowercase THEN lower = TRUE**
- **IF digit THEN digit flag = TRUE**
- **IF special character THEN special = TRUE**
- **END FOR**
-
- **CHECK length >= 8 and no spaces**
-
- **IF all rules satisfied**
- **DISPLAY "VALID PASSWORD"**
- **ELSE**
- **DISPLAY "INVALID PASSWORD"**
- **DISPLAY failed requirements**
- **END IF**
- **END**

7. Python source code and output

```
password = input("Enter a password: ")

upper = False
lower = False
digit = False
special = False

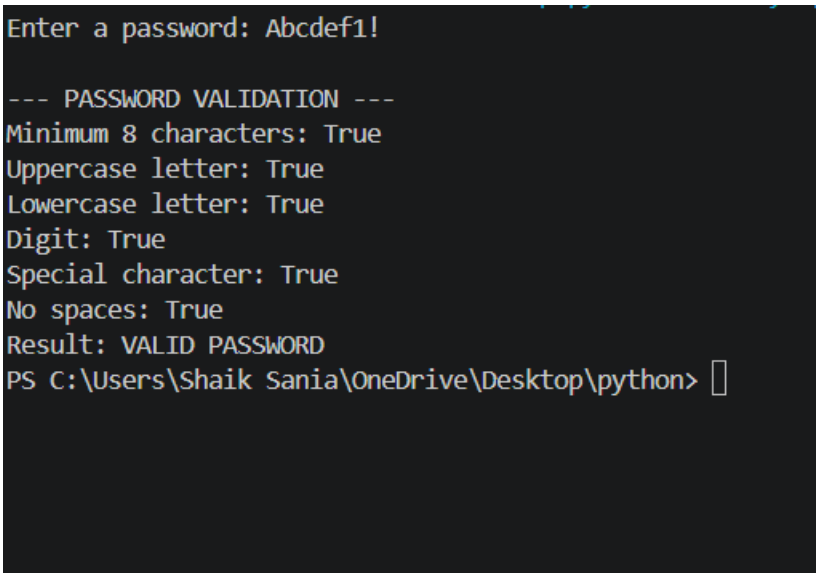
special_chars = "!@#$%^&*"

for ch in password:
    if ch.isupper():
        upper = True
    elif ch.islower():
        lower = True
    elif ch.isdigit():
        digit = True
    elif ch in special_chars:
        special = True

length_ok = len(password) >= 8
space_ok = " " not in password

print("\n--- PASSWORD VALIDATION ---")
print("Minimum 8 characters:", length_ok)
print("Uppercase letter:", upper)
print("Lowercase letter:", lower)
print("Digit:", digit)
print("Special character:", special)
print("No spaces:", space_ok)

if (length_ok and upper and lower and digit
        and special and space_ok):
    print("Result: VALID PASSWORD")
else:
    print("Result: INVALID PASSWORD")
```



8. Test cases

Test input	Expected result
Abcdef1!	Valid
abcdefgh	Invalid
ABCDEFG1!	Invalid: no lowercase
Abcdefgh!	Invalid: no digit
Abcde 1!	Invalid: contains a space

9. Result and conclusion

- **The program checks password requirements by examining individual characters and applying Boolean conditions. It helps demonstrate strings, loops, and logical operators.**

Case Study 3: String-Based Examination Answer Evaluation

1. Problem statement

Develop a Python program that evaluates objective examination answers by comparing a student's responses with a predefined answer key. The program must calculate the number of correct, incorrect, and unanswered questions and determine the final score.

2. Objectives

- **Compare strings using loops and conditions.**
- **Use list indexing to access answers.**
- **Calculate examination scores.**
- **Handle uppercase and lowercase responses consistently.**
- **Display a question-wise evaluation report.**

3. Assumptions

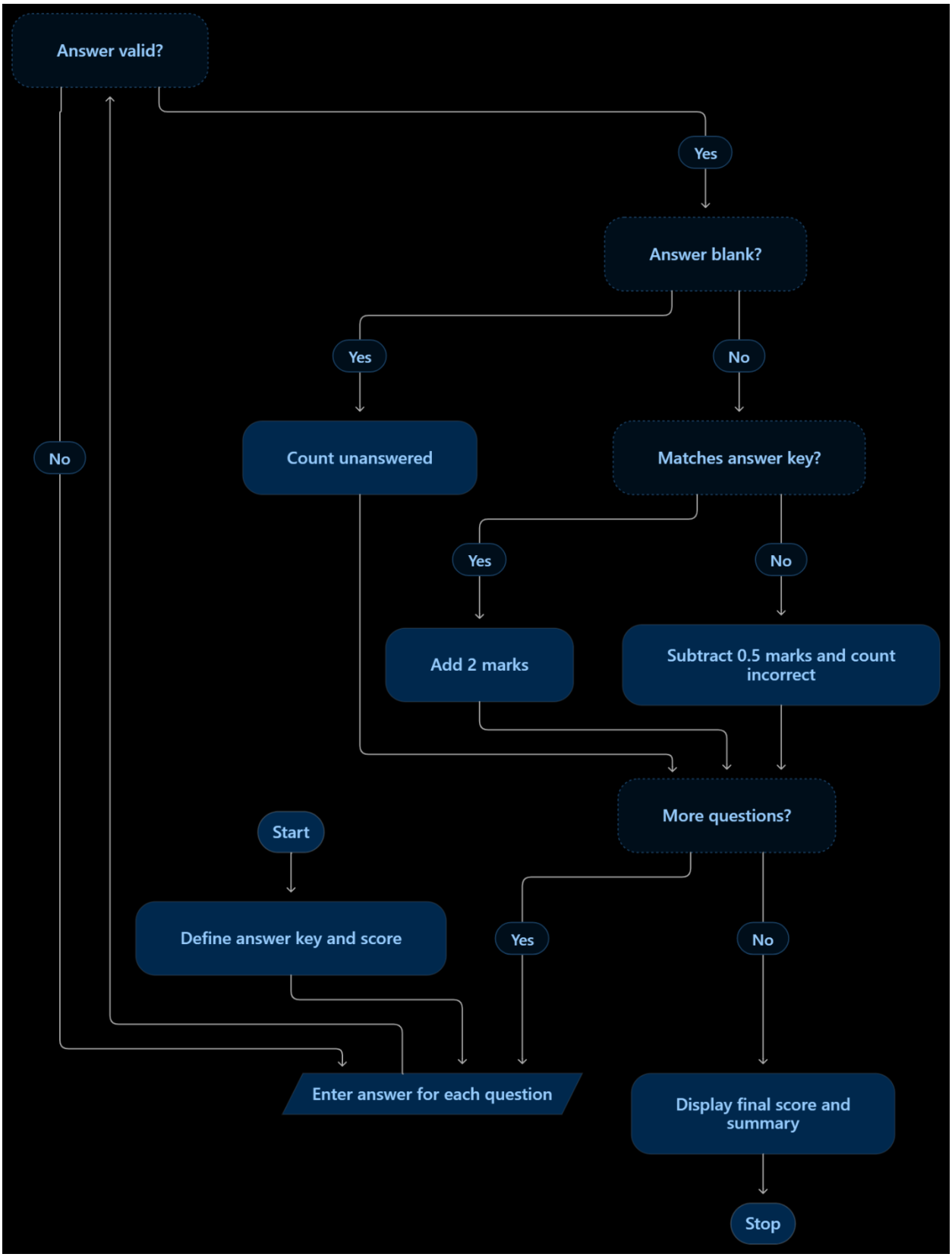
- **The examination contains five multiple-choice questions.**
- **Each correct answer earns 2 marks.**
- **Each incorrect answer loses 0.5 marks.**
- **An unanswered question earns 0 marks.**
- **Answers are entered as A, B, C, or D; blank input means unanswered.**
- **The minimum score is zero; negative scores are not allowed.**

4. Algorithm

1. **Start.**
2. **Define the answer key.**
3. **Initialize the score and answer counters.**
4. **Ask the student to enter each answer.**
5. **Convert the answer to uppercase and validate it.**

6. **Compare it with the answer key.**
7. **Update the score and counters.**
8. **Display the question-wise result and final score.**
9. **Stop.**

5. Flowchart


Answer valid?
Yes
Answer blank?
Yes
No
No
Count unanswered
Matches answer key?
Yes
No
Add 2 marks
Subtract 0.5 marks and count incorrect
More questions?
Start
Define answer key and score
Yes
No
Enter answer for each question
Display final score and summary
Stop

6. Pseudocode

```
BEGIN
  DEFINE answer     key
  SET score = 0
  SET correct = 0, incorrect = 0, unanswered = 0

  FOR each question
    INPUT answer
    CONVERT answer TO uppercase

    WHILE answer is invalid
      INPUT answer again
    END WHILE

    IF answer is blank
      unanswered = unanswered + 1
    ELSE IF answer equals answer            key
      score = score + 2
      correct = correct + 1
    ELSE
      score = score - 0.5
      incorrect = incorrect + 1
    END IF
  END FOR

  SET score = maximum of score and zero
```

DISPLAY results

END

7. Python source code and output

```
answer_key = ["A", "C", "B", "D", "A"]
total_questions = len(answer_key)

score = 0
correct = 0
incorrect = 0
unanswered = 0

print("Enter A, B, C, D, or press Enter to skip.")

for i in range(total_questions):
    while True:
        answer = input(
            f"Question {i + 1}: "
        ).strip().upper()

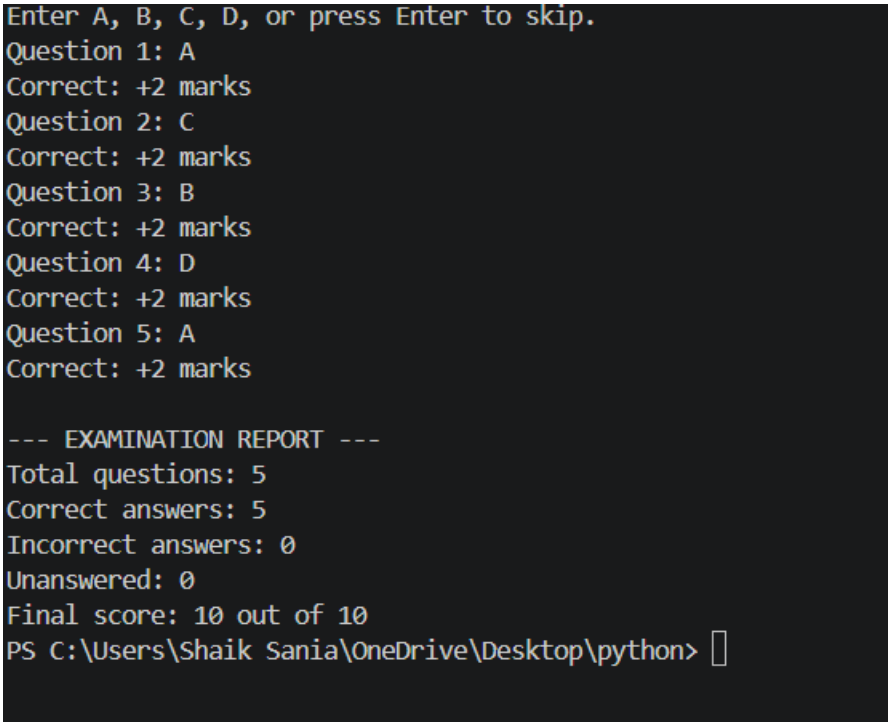
        if answer in ["A", "B", "C", "D", ""]:
            break

        print("Invalid answer. Enter A, B, C, D, or blank.")

    if answer == "":
        unanswered += 1
        print("Unanswered")
    elif answer == answer_key[i]:
        correct += 1
        score += 2
        print("Correct: +2 marks")
    else:
        incorrect += 1
        score -= 0.5
        print("Incorrect: -0.5 marks")

score = max(0, score)

print("\n--- EXAMINATION REPORT ---")
print("Total questions:", total_questions)
print("Correct answers:", correct)
print("Incorrect answers:", incorrect)
print("Unanswered:", unanswered)
print("Final score:", score, "out of", total_questions * 2)
```



8. Test cases

Test	Answers entered	Expected result
1	A, C, B, D, A	10 marks
2	B, B, B, B, B	0.5 marks
3	Blank for all five	0 marks
4	A, C, blank, D, A	8 marks
5	X entered	Rejected; re-enter answer

For Test 2, four incorrect answers and one correct answer give a raw score of \(2 - 4(0.5)=0\). Therefore, the expected score is 0 marks, not 0.5. The scoring rule never permits a negative final score.

9. Result and conclusion

The program successfully evaluates objective answers by comparing strings with an answer key. It calculates scores and tracks correct, incorrect, and unanswered responses.

Case Study 4: Student Marks Matrix Analysis

1. Problem statement

Develop a Python program to store and analyze the marks of multiple students across multiple subjects using a two-dimensional list (matrix). The program must calculate each student's total marks, average, and grade, along with subject-wise averages and the highest-performing student.

2. Objectives

- **Understand two-dimensional lists and matrices.**
- **Use nested loops to process rows and columns.**
- **Calculate totals and averages.**
- **Assign grades using conditional statements.**
- **Identify the highest-performing student.**

3. Assumptions

- **The program uses three students and three subjects.**
- **Each subject is scored out of 100.**
- **Valid marks range from 0 to 100.**
- **Grades are assigned according to the student's average.**

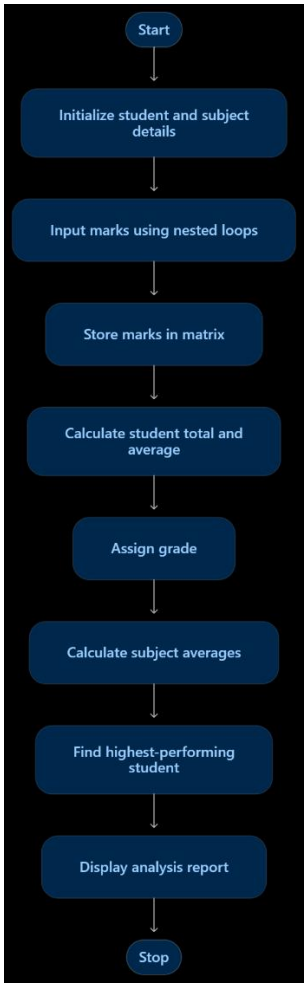
Average	Grade
90–100	A
75–89.99	B
60–74.99	C
40–59.99	D
Below 40	F

4. Algorithm

1. **Start.**

2. **Read the number of students and subjects.**
3. **Enter valid marks for each student using nested loops.**
4. **Store the marks in a two-dimensional list.**
5. **Calculate each student's total and average.**
6. **Assign a grade based on the average.**
7. **Calculate the average marks for each subject.**
8. **Identify the student with the highest total.**
9. **Display the analysis report.**
10. **Stop.**

5. Flowchart



6. Pseudocode

```
BEGIN
  DEFINE student names and subjects
  CREATE empty marks matrix

  FOR each student
    FOR each subject
      INPUT valid marks
      STORE marks
    END FOR
  END FOR

  FOR each student
    CALCULATE total and average
    ASSIGN grade
  END FOR

  FOR each subject
    CALCULATE subject average
  END FOR

  FIND student with highest total
  DISPLAY all results
END
```

7. Python source code and output

```python
# Student Marks Matrix Analysis
students = ["Anu", "Balu", "Charan"]
subjects = ["Python", "Maths", "English"]
marks = []

# Input marks
for student in students:
    row = []
    print("\nEnter marks for", student)
    for subject in subjects:
        mark = int(input(subject + ": "))
        row.append(mark)
    marks.append(row)

# Display student results
print("\n--- STUDENT REPORT ---")
for i in range(len(students)):
    total = sum(marks[i])
    average = total / len(subjects)

    if average >= 90:
        grade = "A"
    elif average >= 75:
        grade = "B"
    elif average >= 60:
        grade = "C"
    elif average >= 40:
        grade = "D"
    else:
        grade = "F"

    print("\nStudent:", students[i])
    print("Marks:", marks[i])
    print("Total:", total)
    print("Average:", round(average, 2))
    print("Grade:", grade)

# Find the highest total
totals = []
for row in marks:
    totals.append(sum(row))

top = totals.index(max(totals))

print("\nTop Student:", students[top])
print("Highest Total:", totals[top])
```

```
Enter marks for Anu
Python: 100
Maths: 100
English: 100

Enter marks for Balu
Python: 100
Maths: 90
English: 90

Enter marks for Charan
Python: 80
Maths: 90
English: 100

--- STUDENT REPORT ---

Student: Anu
Marks: [100, 100, 100]
Total: 300
Average: 100.0
Grade: A

Student: Balu
Marks: [100, 90, 90]
Total: 280
Average: 93.33
Grade: A

Student: Charan
Marks: [80, 90, 100]
Total: 270
Average: 90.0
Grade: A

Top Student: Anu
Highest Total: 300
PS C:\Users\Shaik Sania\OneDrive\Desktop\python>
```

8. Test cases

Test	Input or condition	Expected result
1	All marks are 100	All students receive grade A
2	All marks are 0	All students receive grade F
3	Marks are 75, 80, 85	Average 80; grade B
4	Marks are -5 or 105	Input rejected
5	Different totals for each student	Student with highest total identified

9. Result and conclusion

The program successfully stores marks in a matrix and performs student-wise and subject-wise analysis. It demonstrates nested loops, lists, indexing, arithmetic operations, and conditional statements.

Case Study 5: Weekly Electricity Consumption Matrix

1. Problem statement

Develop a Python program to record the daily electricity consumption of multiple households for seven days. Store the readings in a two-dimensional matrix, calculate each household's weekly consumption and daily averages, and identify the highest and lowest consuming households.

2. Objectives

- **Store weekly readings in a matrix.**
- **Use nested loops to collect and process data.**
- **Calculate weekly totals and daily averages.**
- **Compare household consumption.**
- **Identify the maximum and minimum consumption values.**

3. Assumptions

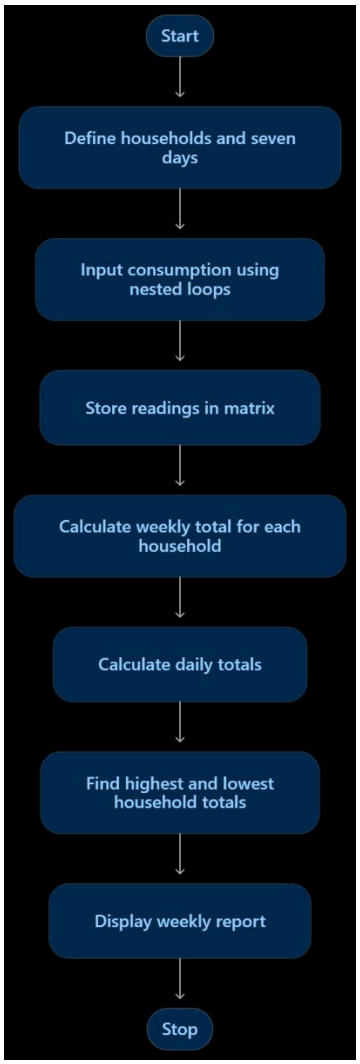
- **The program records three households for seven days.**
- **Consumption is measured in kilowatt-hours (kWh).**
- **Each reading must be non-negative.**
- **Each row represents one household; each column represents one day.**
- **The sample data is illustrative and is not an actual electricity bill.**

4. Algorithm

1. **Start.**
2. **Define the household names and seven days.**
3. **Input daily electricity consumption using nested loops.**
4. **Store readings in a two-dimensional list.**
5. **Calculate weekly consumption for each household.**
6. **Calculate the average daily consumption.**
7. **Calculate the total consumption for each day.**

8. **Find the highest and lowest consuming households.**
9. **Display the weekly matrix and summary.**
10. **Stop.**

5. Flowchart



6. Pseudocode

BEGIN

DEFINE households and seven days

CREATE empty consumption matrix

```
  FOR each household
    FOR each day
      INPUT non-negative consumption
      STORE reading
    END FOR
  END FOR

  FOR each household
    CALCULATE weekly total
    CALCULATE daily average
  END FOR

  FOR each day
    CALCULATE total consumption
  END FOR

  FIND highest and lowest household totals
  DISPLAY weekly report
END
```

7. Python source code and output

```python
# Weekly Electricity Consumption Matrix

houses = ["House A", "House B", "House C"]
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
data = []

# Input electricity consumption
for house in houses:
    row = []
    print("\nEnter units for", house)

    for day in days:
        units = float(input(day + ": "))
        row.append(units)

    data.append(row)

# Display weekly totals
totals = []

print("\n--- WEEKLY REPORT ---")

for i in range(len(houses)):
    total = sum(data[i])
    average = total / 7
    totals.append(total)

    print(houses[i], "Total:", total, "Average:", round(average, 2))

# Find highest and lowest consumption
print("Highest consumption:", houses[totals.index(max(totals))])
print("Lowest consumption:", houses[totals.index(min(totals))])
print("Total units used:", sum(totals), "kWh")
```

```
Enter units for House A
Mon: 10
Tue: 10
Wed: 10
Thu: 10
Fri: 10
Sat: 10
Sun: 10

Enter units for House B
Mon: 20
Tue: 10
Wed: 30
Thu: 40
Fri: 39
Sat: 20
Sun: 30

Enter units for House C
Mon: 0
Tue: 0
Wed: 0
Thu: 0
Fri: 0
Sat: 0
Sun: 0

--- WEEKLY REPORT ---
House A Total: 70.0 Average: 10.0
House B Total: 189.0 Average: 27.0
House C Total: 0.0 Average: 0.0
Highest consumption: House B
Lowest consumption: House C
Total units used: 259.0 kWh
PS C:\Users\Shaik Sania\OneDrive\Desktop\python>
```

8. Test cases

Test	Input or condition	Expected result
1	All readings are 0	All weekly totals are 0
2	All readings are 10	Each household totals 70 kWh
3	One household has higher readings	It is identified as highest consuming
4	One household has lower readings	It is identified as lowest consuming
5	Negative consumption	Input rejected

9. Result and conclusion

The program successfully records and analyzes electricity consumption for multiple households over seven days. It demonstrates two-dimensional lists, nested loops, totals, averages, and maximum/minimum calculations.

Case Study 6: Student Email and Identifier Validation

1. Problem statement

Develop a Python program to validate a student's email address and student ID. The program checks whether the email contains @ and a dot (.), and whether the student ID follows a predefined format.

2. Objectives

- **Understand string operations.**
- **Apply if-else conditions.**
- **Validate email addresses and student IDs.**
- **Display appropriate validation messages.**

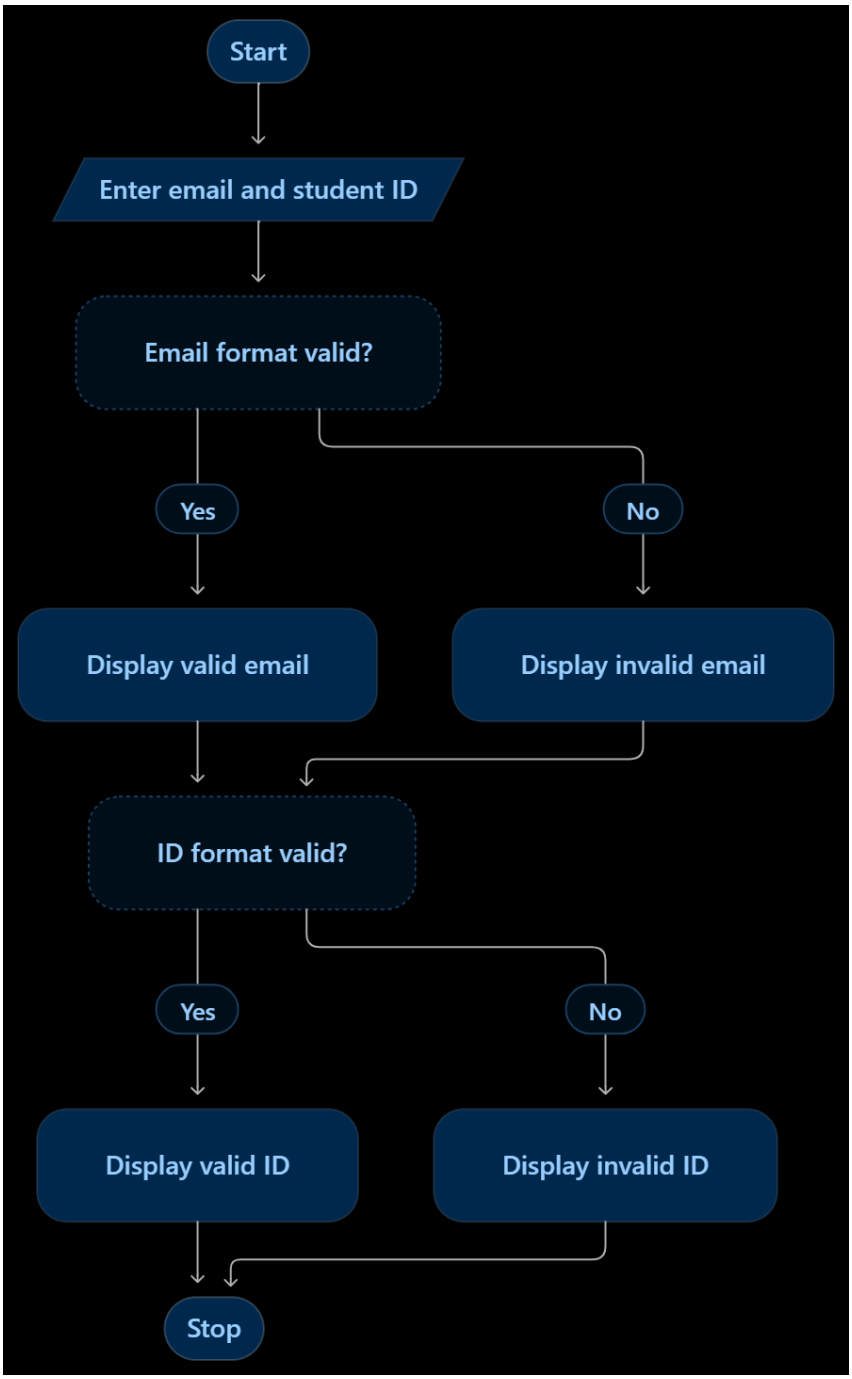
3. Assumptions

- **The email must contain @ and ..**
- **The email must not contain spaces.**
- **The student ID must start with STU followed by exactly three digits, such as STU101.**
- **These are basic format checks, not complete email verification.**

4. Algorithm

1. **Start.**
2. **Input the email address and student ID.**
3. **Check the email format.**
4. **Check whether the ID starts with STU and has three digits after it.**
5. **Display whether each input is valid.**
6. **Stop.**

5. Flowchart


Start
Enter email and student ID
Email format valid?
Yes
No
Display valid email
Display invalid email
ID format valid?
Yes
No
Display valid ID
Display invalid ID
Stop

6. Pseudocode

```
BEGIN
  INPUT email
  INPUT student_id

  IF email contains @ and .
    AND email has no spaces
     DISPLAY "Valid email"
  ELSE
     DISPLAY "Invalid email"
  END IF

  IF ID starts with STU
    AND remaining characters are 3 digits
     DISPLAY "Valid student ID"
  ELSE
     DISPLAY "Invalid student ID"
  END IF
END
```

7. Python source code and output

```python
email = input("Enter student email: ")
student_id = input("Enter student ID: ")

if "@" in email and "." in email and " " not in email:
    print("Valid email")
else:
    print("Invalid email")

if (student_id.startswith("STU")
        and len(student_id) == 6
        and student_id[3:].isdigit()):
    print("Valid student ID")
else:
    print("Invalid student ID")
```

```
Enter student email: anu@gmail.com
Enter student ID: STU101
Valid email
Valid student ID
PS C:\Users\Shaik Sania\OneDrive\Desktop\python>
```

8. Test cases

Email	Student ID	Expected result
anu@gmail.com	STU101	Both valid
anugmail.com	STU101	Invalid email
balu@gmail.com	ABC101	Invalid ID
charan@gmail.com	STU12A	Invalid ID

9. Result and conclusion

The program validates basic student email and ID formats using string operations and conditional statements. It demonstrates simple input validation.

Case Study 7: Nested Loop Calendar Generator

1. Problem statement

Develop a Python program to generate a monthly calendar using nested loops. The program accepts the number of days in a month and the starting weekday, then displays the dates in a calendar format.

2. Objectives

- **Understand nested for loops.**
- **Use conditional statements.**
- **Arrange dates in rows and columns.**
- **Generate a simple monthly calendar.**

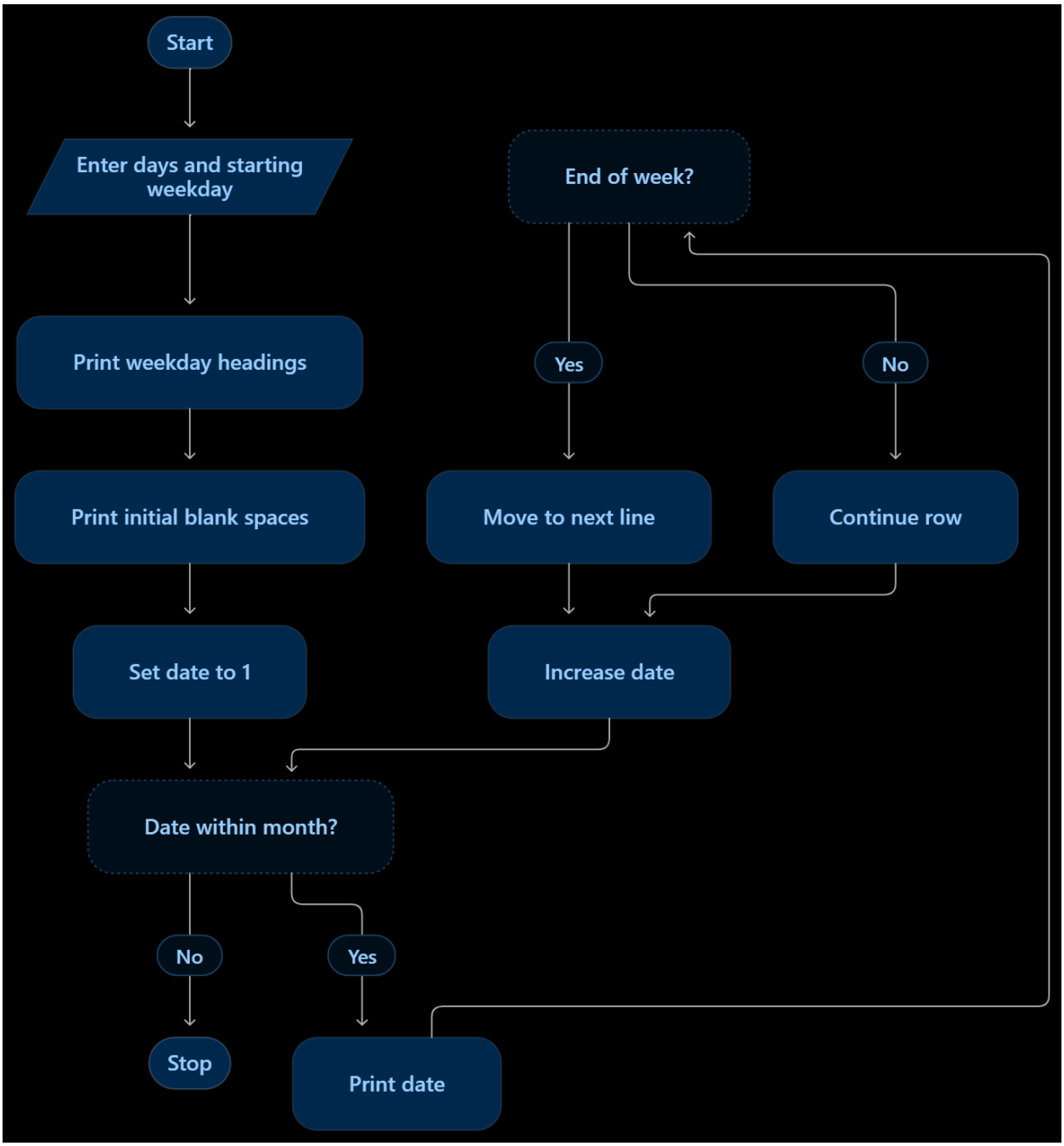
3. Assumptions

- **The month has 28 to 31 days.**
- **The user enters the starting weekday as a number from 0 to 6.**
- **Monday = 0, Tuesday = 1, Wednesday = 2, Thursday = 3, Friday = 4, Saturday = 5, Sunday = 6.**
- **The program does not determine the month or year automatically.**

4. Algorithm

1. **Start.**
2. **Input the number of days and starting weekday.**
3. **Print the weekday headings.**
4. **Print blank spaces before the first date.**
5. **Use a loop to print each date.**
6. **Move to a new line after Sunday.**
7. **Stop.**

5. Flowchart


Start
Enter days and starting weekday
Print weekday headings
Print initial blank spaces
Set date to 1
Date within month?
No
Stop
Yes
Print date
Increase date
End of week?
Yes
Move to next line
No
Continue row

6. Pseudocode

```
BEGIN
  INPUT days
  INPUT start_day

  PRINT weekday headings

  PRINT blank spaces for start_day

  FOR date FROM 1 TO days
    PRINT date
    IF (date + start_day) is divisible by 7
      MOVE TO NEXT LINE
    END IF
  END FOR
END
```

7. Python source code and output

```python
days = int(input("Enter days in month (28-31): "))
start = int(input("Enter starting weekday (Mon=0, Sun=6): "))

if 28 <= days <= 31 and 0 <= start <= 6:
    print("\nMon Tue Wed Thu Fri Sat Sun")

    for i in range(start):
        print("    ", end="")

    for date in range(1, days + 1):
        print(f"{date:3}", end=" ")

        if (date + start) % 7 == 0:
            print()

    print()
else:
    print("Invalid input")
```

```
Enter days in month (28-31): 30
Enter starting weekday (Mon=0, Sun=6): 5

Mon Tue Wed Thu Fri Sat Sun
                      1   2
  3   4   5   6   7   8   9
 10  11  12  13  14  15  16
 17  18  19  20  21  22  23
 24  25  26  27  28  29  30

PS C:\Users\Shaik Sania\OneDrive\Desktop\python>
```

8. Test cases

Days	Starting weekday	Expected result
30	0	Calendar begins Monday
31	5	Calendar begins Saturday

28	**6**	**Calendar begins Sunday**
32	**0**	**Invalid input**
30	**8**	**Invalid input**

9. Result and conclusion

The program generates a monthly calendar using loops and conditional statements. It demonstrates how dates can be arranged into weekly rows.

Case Study 8: Nested List Classroom Occupancy Analysis

1. Problem statement

Develop a Python program to analyze classroom occupancy using a nested list. The program records the number of students in three classrooms over three sessions and calculates the total attendance for each classroom.

2. Objectives

- **Understand nested lists.**
- **Use nested loops to process data.**
- **Calculate row totals and averages.**
- **Identify the classroom with the highest attendance.**

3. Assumptions

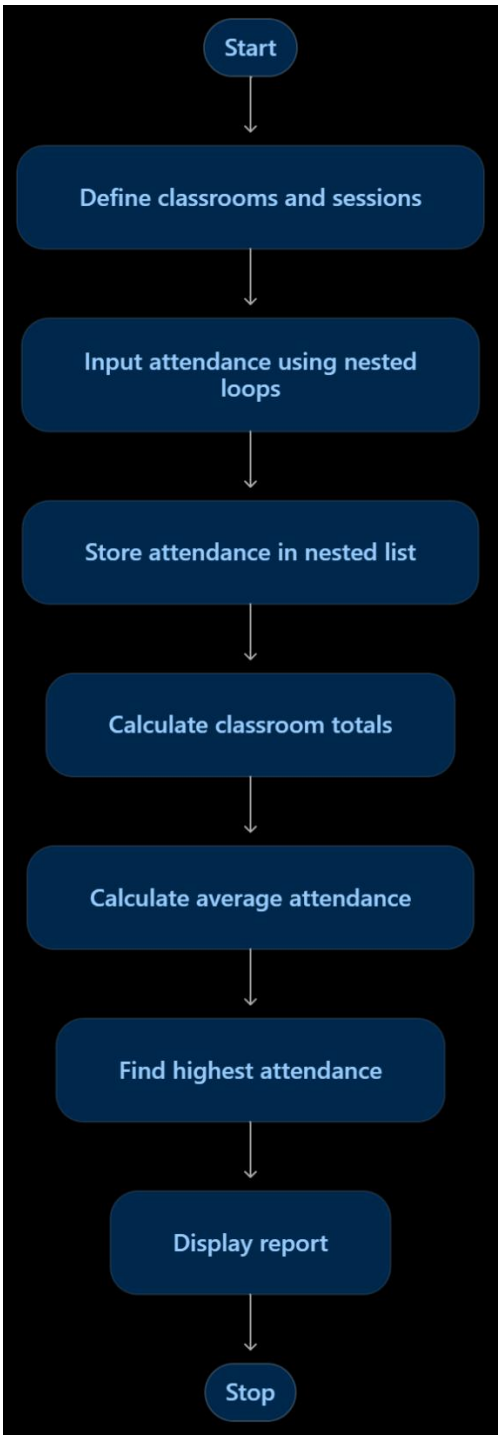
- **There are three classrooms and three sessions.**
- **Each entry represents the number of students present in one session.**
- **Attendance values must be non-negative whole numbers.**
- **Each classroom has a capacity of 60 students.**

4. Algorithm

1. **Start.**
2. **Define the classroom names and sessions.**
3. **Input attendance for each classroom and session.**
4. **Store the values in a nested list.**
5. **Calculate each classroom's total attendance.**
6. **Calculate the average attendance per session.**

7. **Find the classroom with the highest total.**
8. **Display the report.**
9. **Stop.**

5. Flowchart



6. Pseudocode

```
BEGIN
  DEFINE classrooms and sessions
  CREATE empty attendance list

  FOR each classroom
    CREATE empty row
    FOR each session
      INPUT attendance
      ADD attendance to row
    END FOR
    ADD row to attendance list
  END FOR

  FOR each classroom
    CALCULATE total and average
  END FOR

  FIND classroom with highest total
  DISPLAY report
END
```

7. Python source code and output

```
rooms = ["Room A", "Room B", "Room C"]
sessions = ["Morning", "Afternoon", "Evening"]
attendance = []

for room in rooms:
    row = []
    print("\nEnter attendance for", room)

    for session in sessions:
        count = int(input(session + ": "))
        row.append(count)

    attendance.append(row)

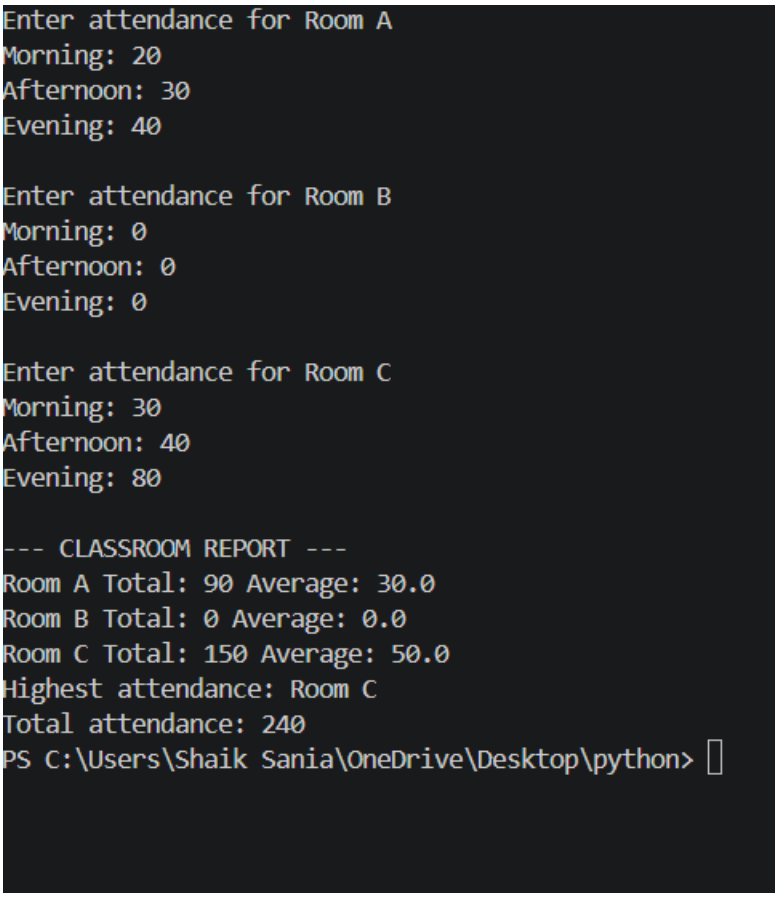
totals = []

print("\n--- CLASSROOM REPORT ---")

for i in range(len(rooms)):
    total = sum(attendance[i])
    average = total / len(sessions)
    totals.append(total)

    print(rooms[i], "Total:", total,
          "Average:", round(average, 2))

top = totals.index(max(totals))
print("Highest attendance:", rooms[top])
print("Total attendance:", sum(totals))
```



8. Test cases

Test	Attendance input	Expected result
1	All values are 0	All totals are 0
2	Room A: 20, 30, 40	Total = 90, average = 30
3	Room B has the highest total	Room B is identified
4	All rooms have equal totals	First matching room is displayed as highest

9. Result and conclusion

The program stores classroom attendance in a nested list and calculates totals and averages. It demonstrates lists, nested loops, and basic data analysis.

Note: This beginner version assumes the user enters valid non-negative attendance counts that do not exceed classroom capacity. Input validation can be added as an improvement.

Case Study 9: Inventory Matrix Reconciliation

1. Problem statement

Develop a Python program to compare the recorded stock of items with the actual stock available in a store. The program calculates the difference between recorded and actual quantities and identifies whether stock is missing, extra, or matching.

2. Objectives

- **Store inventory information in lists.**
- **Compare two sets of stock quantities.**
- **Calculate stock differences.**
- **Identify missing and excess stock.**

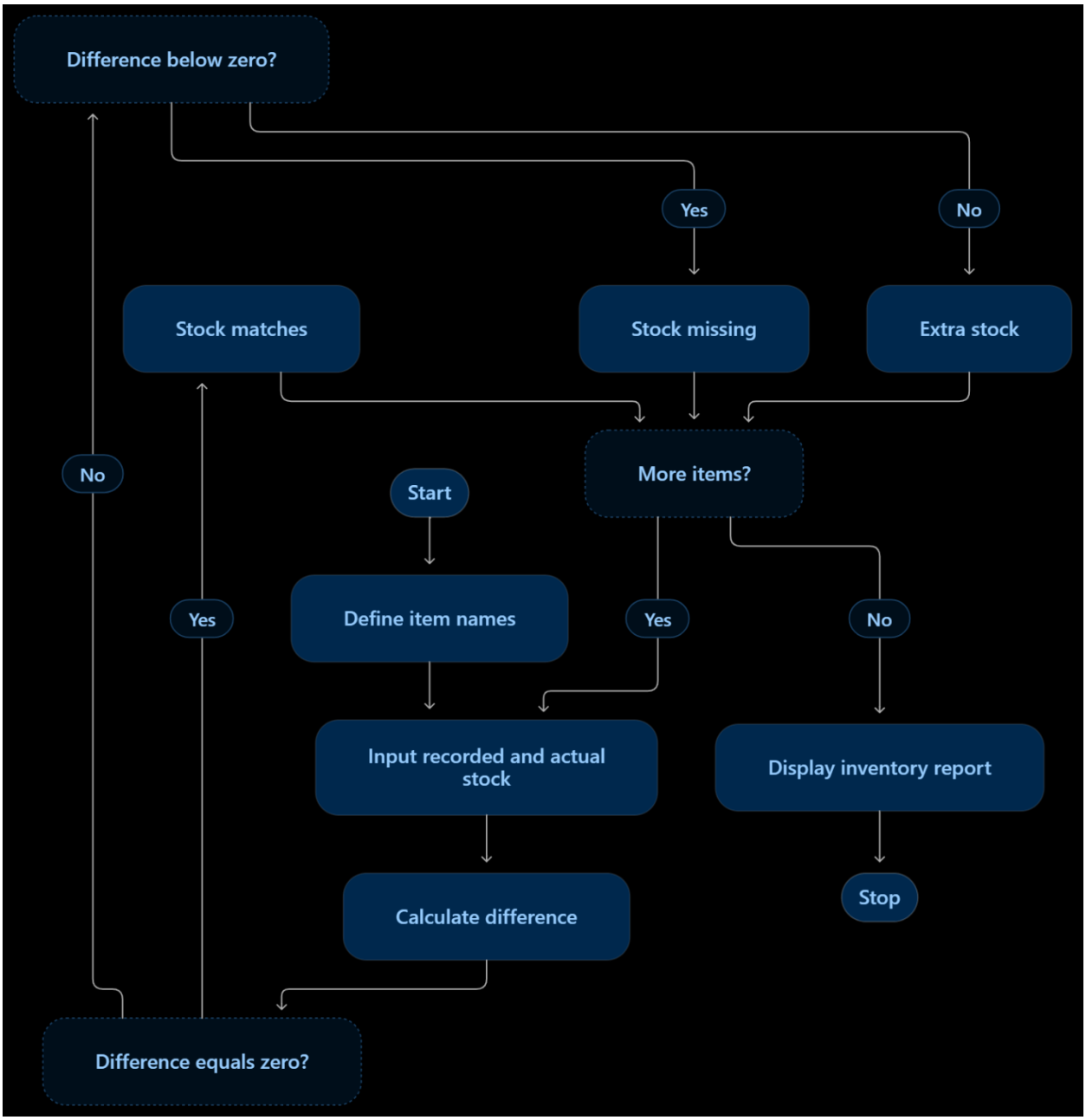
3. Assumptions

- **The store has four items.**
- **Recorded and actual stock quantities are non-negative integers.**
- **Difference = Actual stock – Recorded stock.**
- **A negative difference means stock is missing; a positive difference means extra stock.**

4. Algorithm

1. **Start.**
2. **Define item names.**
3. **Enter recorded and actual stock quantities.**
4. **Calculate the difference for each item.**
5. **Identify whether stock is matching, missing, or extra.**
6. **Display the inventory report.**
7. **Stop.**

5. Flowchart



6. Pseudocode

BEGIN

DEFINE item names

FOR each item

INPUT recorded stock

INPUT actual stock

difference = actual - recorded

IF difference equals zero

DISPLAY "Stock matches"

ELSE IF difference is negative

DISPLAY "Stock missing"

ELSE

DISPLAY "Extra stock"

END IF

END FOR

END

7. Python source code and output

```
items = ["Books", "Pens", "Bags", "Pencils"]

for item in items:
    print("\nItem:", item)

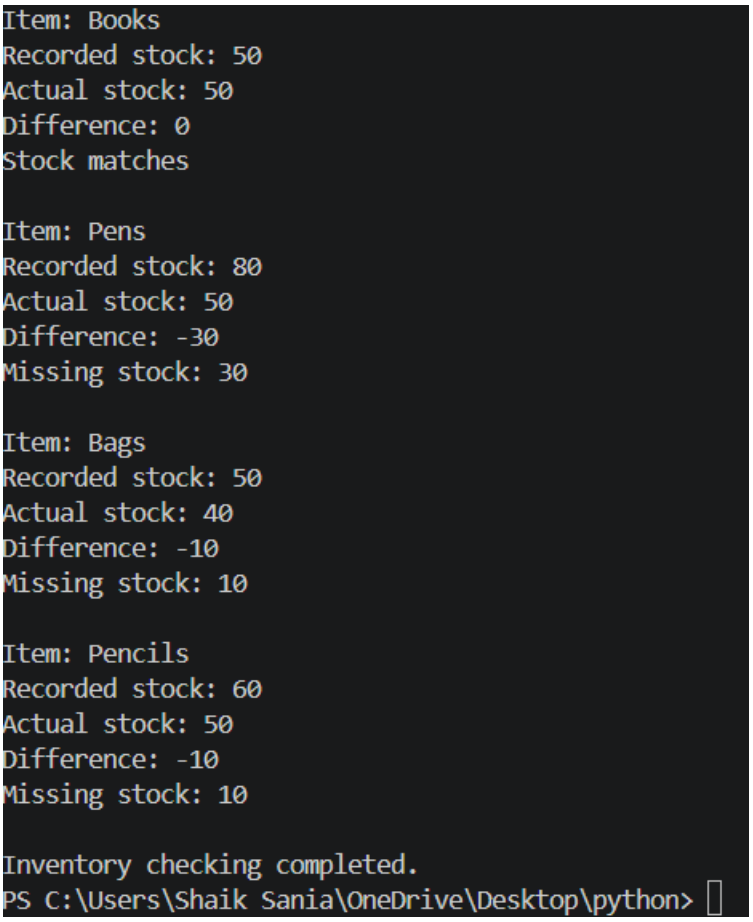
    recorded = int(input("Recorded stock: "))
    actual = int(input("Actual stock: "))

    difference = actual - recorded

    print("Difference:", difference)

    if difference == 0:
        print("Stock matches")
    elif difference < 0:
        print("Missing stock:", abs(difference))
    else:
        print("Extra stock:", difference)

print("\nInventory checking completed.")
```



8. Test cases

Recorded	Actual	Expected result
50	50	Stock matches
50	40	Missing stock: 10
50	60	Extra stock: 10
0	10	Extra stock: 10
20	0	Missing stock: 20

9. Result and conclusion

The program compares recorded and actual inventory quantities and identifies discrepancies. It demonstrates loops, arithmetic operations, and conditional statements.

Case Study 5: Library Borrowing and Fine Reconciliation

1. Problem statement

Develop a Python program to calculate library fines for borrowed books. The program accepts the number of days a book is overdue and calculates the fine based on a fixed charge per overdue day.

2. Objectives

- **Understand input and output operations.**
- **Apply if-else statements.**
- **Calculate fines using arithmetic operations.**
- **Process multiple library transactions using a loop.**

3. Assumptions

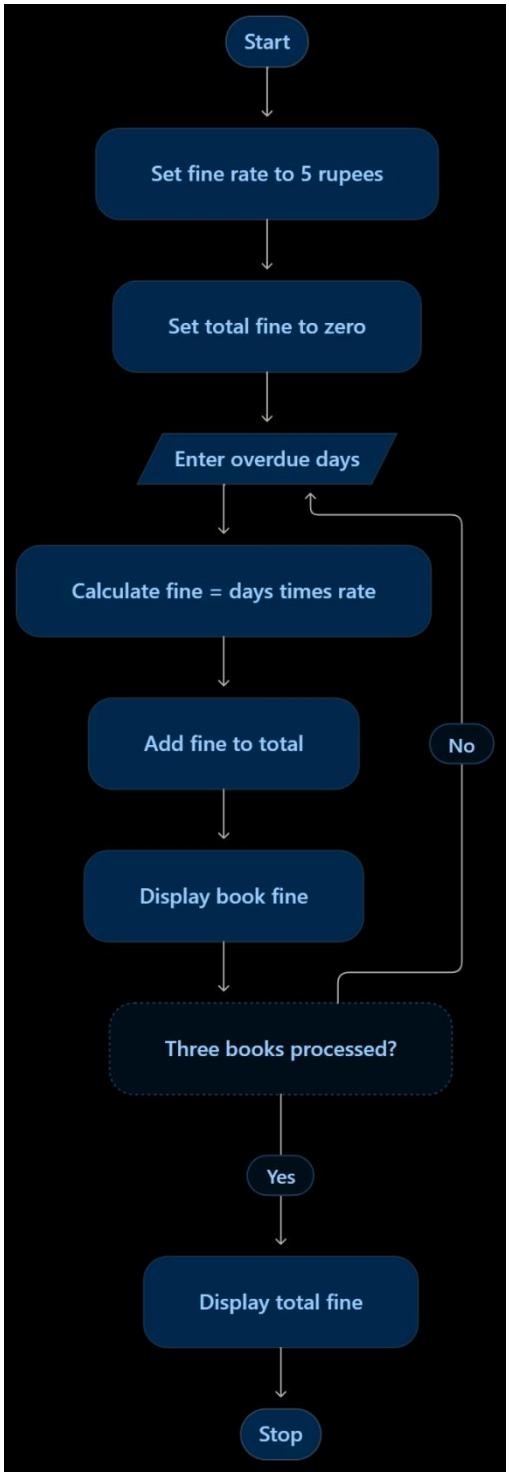
- **The library processes three borrowed books.**
- **Books returned on or before the due date have no fine.**
- **The fine is ₹5 per overdue day.**
- **The number of overdue days must be a non-negative integer.**
- **The program does not calculate loan periods or different fine rates for different book types.**

4. Algorithm

1. **Start.**
2. **Set the fine rate to ₹5 per day.**
3. **Repeat for three books.**
4. **Input the number of overdue days.**
5. **Calculate the fine.**

6. **Add the fine to the total fine.**
7. **Display the fine for each book.**
8. **Display the total fine.**
9. **Stop.**

5. Flowchart



6. Pseudocode

BEGIN

SET fine rate = 5

SET total line = 0

FOR each of 3 books

INPUT overdue days

fine = overdue days * fine _rate

total fine = total fine + fine

DISPLAY fine

END FOR

DISPLAY total fine

END

7. Python source code and output

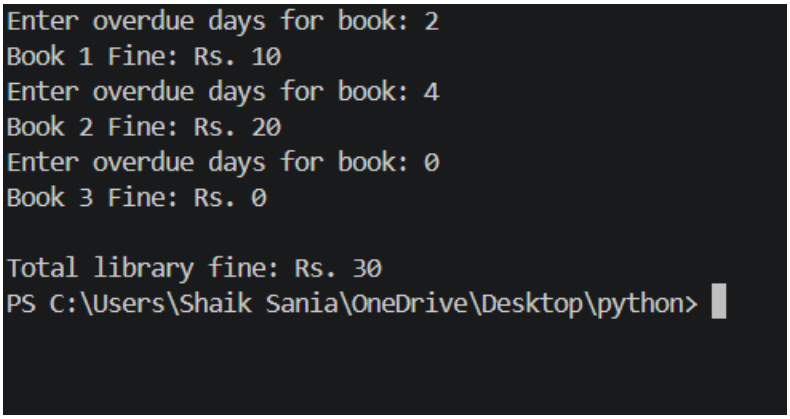
```
fine_rate = 5
total_fine = 0

for book in range(1, 4):
    days = int(input("Enter overdue days for book: "))

    fine = days * fine_rate
    total_fine = total_fine + fine

    print("Book", book, "Fine: Rs.", fine)

print("\nTotal library fine: Rs.", total_fine)
```



8. Test cases

Overdue days	Expected fine
0	₹0
1	₹5
5	₹25
10	₹50
3 books overdue by 2, 4, and 0 days	Total ₹30

9. Result and conclusion

The program calculates individual book fines and the total fine for multiple books. It demonstrates loops, variables, and arithmetic operations.